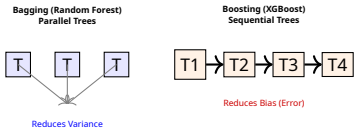
XGBoost: Regression & Classification

Extreme Gradient Boosting: From Theory to Production

1 Introduction to XGBoost

Definition XGBoost (Extreme Gradient Boosting) is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable.

XGBoost vs. Others



- **Decision Tree**: Single weak learner. High variance.
- **Random Forest**: Bagging (Parallel). Reduces variance.
- **Gradient Boosting (GBM)**: Boosting (Sequential). Reduces bias.
- **XGBoost**: GBM + **System Optimization** (Parallelization, Cache-aware) + **Algorithmic Enhancements** (Regularization, Handling Sparsity).

2 Core Concepts

Boosting Intuition Start with a weak model. Add a new model that fixes the errors (residuals) of the previous one. Repeat.

Objective Function

$$\text{Obj}(\theta) = L(\theta) + \Omega(\theta)$$

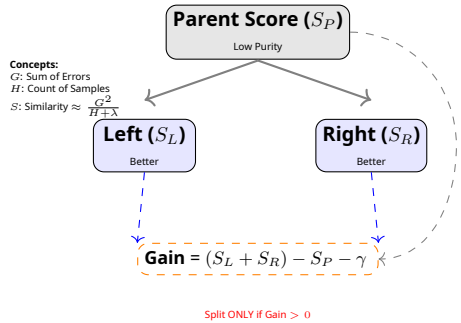
- $L(\theta)$: Training Loss (MSE, LogLoss).
- $\Omega(\theta)$: Regularization (Complexity penalty).

Key Innovations

- **Second-Order Approximation**: Uses Taylor expansion (Gradient g + Hessian h) for faster convergence.
- **Regularization**: L1 (α) and L2 (λ) built-in.

3 Architecture & Math

Split Finding (The "Gain" Metric) Instead of Gini/Entropy, XGBoost uses Similarity Scores derived from Gradients (g) and Hessians (h).

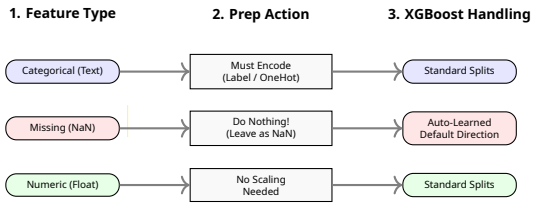


Tree Pruning

- **Pre-stopping**: Stop splitting if Gain < γ (Gamma).
- **Post-pruning**: Grow deep, then prune nodes with negative gain.

4 Data Preparation

Handling Features



- **Categorical**: XGBoost (unlike CatBoost) requires numbers. Use `OneHotEncoder` or `LabelEncoder`. Newer versions allow `enable_categorical=True` but require Pandas category dtype.
- **Missing Values**: Do NOT replace with mean/median unless necessary. XGBoost learns the optimal "default direction" (Left or Right) for NaNs during training.
- **Scaling**: Not required for tree-based models.

DMatrix XGBoost's optimized internal data structure.

```
import xgboost as xgb
dtrain = xgb.DMatrix(X_train, label=y_train)
```

5 XGBoost for Regression

Objective: `reg:squarederror` **Metrics**:

- **RMSE**: Penalizes large errors.
- **MAE**: Robust to outliers.

```
from xgboost import XGBRegressor
reg = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    objective='reg:squarederror'
)
reg.fit(X_train, y_train)
```

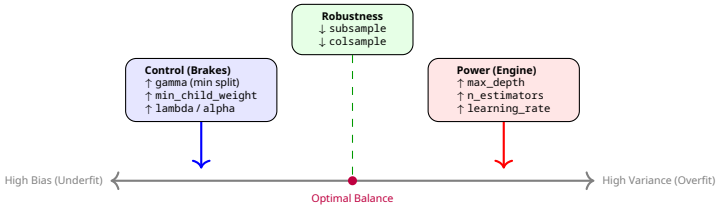
6 XGBoost for Classification

Objectives:

- Binary: `binary:logistic` (Output: Probabilities).
- Multiclass: `multi:softprob` (Output: Prob dist).

```
from xgboost import XGBClassifier
clf = XGBClassifier(
    n_estimators=100,
    objective='binary:logistic',
    eval_metric='logloss'
)
clf.fit(X_train, y_train)
```

7 Key Hyperparameters



Learning Control

- `learning_rate (eta)`: Step size [0.01 - 0.3]. Lower needs more estimators.
- `n_estimators`: Number of trees.

Tree Complexity (Bias/Variance)

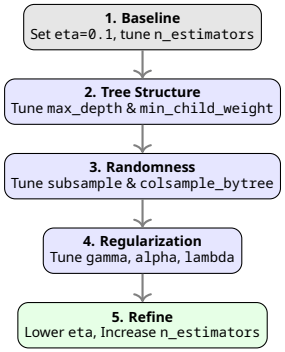
- `max_depth`: Max tree depth [3-10]. Higher = Overfit.
- `min_child_weight`: Min sum of hessians (weight) in a leaf. Higher = Conservative (Underfit).
- `gamma`: Min loss reduction required to split.

Stochastic Sampling

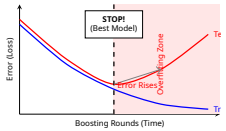
- `subsample`: % of rows per tree.
- `colsample_bytree`: % of columns per tree.

8 Hyperparameter Tuning

Step-by-Step Strategy



Early Stopping (Prevent Overfitting) Stop training when validation error starts to rise.

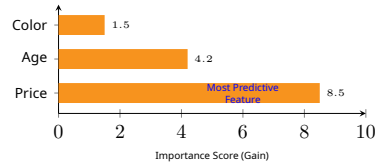



```
# Implementation
model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    early_stopping_rounds=10
)
```

9 Interpretability

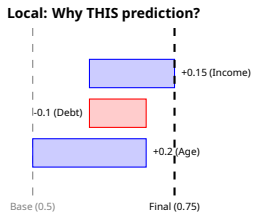
Global Importance (Overall Model) Which features drive the model's decisions?

- **Gain (Quality)**: Average improvement in accuracy when a feature is used. *Best default.*
- **Weight (Frequency)**: How many times a feature appears in trees.
- **Cover (Reach)**: How many observations are affected by this feature.




```
xgb.plot_importance(model, importance_type='gain')
```

Local Interpretability (Per Prediction) Why was *this specific* row classified this way?



SHAP Values: Calculates the contribution of each feature to the prediction (positive or negative pull).

10 Production

Saving/Loading

```
# Save (JSON is widely supported)
model.save_model("model.json")

# Load
loaded = XGBClassifier()
loaded.load_model("model.json")
```

Performance Tips

- **GPU Training**: `tree_method='gpu_hist'`.
- **Inference**: Convert to ONNX or Treelite for faster C++ inference.

11 Full Pipeline Example

```
import pandas as pd
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Load & Split
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2)

# 2. Config
params = {
    'objective': 'binary:logistic',
    'max_depth': 4,
    'learning_rate': 0.05,
    'n_estimators': 500
}

# 3. Train w/ Early Stopping
model = xgb.XGBClassifier(**params)
model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    early_stopping_rounds=50,
    verbose=False
)

# 4. Predict
preds = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, preds)}")
```

verbose=False (Silent Mode)

Common Pitfalls

- **Overfitting**: Check training vs validation loss gap. Increase gamma or `min_child_weight`.
- **Imbalanced Data**: Use `scale_pos_weight` for binary classification.